Errata and clarifications — *Control flow obfuscation with irreducible loops and self-modifying code*

Errata to G. Morse, A. Alqaradaghi and T. Kozsik, *Publ. Math. Debrecen* 100 (Suppl.) (2022) 617–637, DOI 10.5486/PMD.2022.Suppl.5

Each item quotes the published text exactly (*before*), gives the replacement (*after*) and a reason. *Lean*: proved in the accompanying Lean 4 development (declaration names given); *listing*: checked against native runs (evidence, not proof); *editorial*: an internal inconsistency or a primary source. Page numbers are those of the journal. Items marked *open* await the authors' confirmation and are not applied.

Corrections

P01 (p. 619, Section 2, paragraph 1; Lean). Before: `a set of clauses C = {C_1, ..., C_n} where for all i in [1..n]`. After: `a set of clauses C = {C_1, ..., C_m} where for all i in [1..m]`. The index bound n is already the number of variables; the number of clauses is a separate quantity (Fig. 1 has 8 variables and 12 clauses).

P02 (p. 619, Section 2, paragraph 2; Lean). Before: `A clause with one variable has a forced value which can be replaced in all other equations or considered to be a clause with the same variable twice. Regardless it need not effect the implication graph or could be a trivial self-loop.`. After: `A clause with one literal l has a forced value which can be substituted into all other clauses, or the clause can be considered to be (l or l), which yields the single implication (not l) implies l. In the second case the implication is an edge of the implication graph (from the negation of the literal to the literal), not a self-loop, and it must not be dropped: omitting it would make (x) and (not x) together appear satisfiable.`. For (l or l) the two contrapositive implications coincide in the single edge (not l) -> l, which is not a self-loop and carries the constraint; the published sentence says it is unnecessary or trivial.

P03 (p. 619, Section 2, paragraph 3; Lean). Before: `A vertex with two out-edges is a Boolean conditional, otherwise it contains more than two out-edges and is an n-way conditional.`. After: `A vertex with two out-edges is a Boolean conditional, a vertex with more than two out-edges is an n-way conditional, and a vertex with no out-edge is an exit (return) vertex.`. The published classification has no case for out-degree 0 although the paper's own examples contain such vertices (the exit of Fig. 4). *Lean:* `fig4_kinds`.

P04 (p. 623, Section 3.1, last sentence; Lean). Before: `Finally, implications can randomly be added to the forest such that no loops are created which would imply unsatisfiability.`. After: `Finally, implications can randomly be added to the forest, each together with its contrapositive, provided that none leads from a literal that is true under the desired assignment to a literal that is false under it. Loops are allowed (they only merge literals into one component); a loop through a variable and its negation would imply unsatisfiability and cannot arise under this condition.`. Acyclicity is neither necessary (cycles within the true literals are harmless) nor sufficient (an acyclic implication from a true to a false literal makes the desired assignment violate the formula) for the generated formula to have the desired assignment as its solution. *Lean:* `forcing_agree`, `planted_solve`, `counter_violates`, `counter_unsat`, `loop_keeps`, `no_loop_through_negation`.

P05 (p. 625, Section 4, addition; Lean). Before: `s_i = a_i xor b_i xor c_{i+1}`. After: `s_i = a_i xor b_i xor c_i`. c_i is the carry into bit i (c_0 = 0, c_{i+1} is produced from bit i); with c_{i+1} the sum bit would depend on the carry out, e.g. a = b = 1 gives s_0 = 1 xor 1 xor 1 = 1 instead of 0. *Lean:* `published_add_fails`, `addBit_post`, `ripple_add`.

P06 (p. 624, Section 4 (p. 624) and Section 4.1 (p. 625); Lean). Before: `using logic synthesis to construct an optimal circuit is NP-hard based on the circuit SAT problem.` After: `constructing a smallest circuit for a given truth table is the Minimum Circuit Size Problem (MCSP) (Kabanets and Cai, STOC 2000): it is in NP, but whether it is NP-hard is a well-known open problem (it is NP-hard for DNF formulas (Allender, Hellerstein, McCabe, Pitassi and Saks, SIAM J. Comput. 38 (2008)) and, under randomized reductions, for circuits with several outputs (Ilango, Loff and Oliveira, CCC 2020)). Circuit satisfiability, which is NP-complete, only asks whether a given circuit has an input that makes it output 1 and does not by itself give the hardness of finding an optimal circuit; minimizing a given circuit (finding an equivalent circuit of at most a given size) is coNP-hard, since the circuits that compute the constant 0 function are exactly the unsatisfiable ones, which a one-gate circuit replaces.` Three distinct problems are conflated: circuit satisfiability (NP-complete), minimizing a given circuit (coNP-hard) and constructing a smallest circuit from a truth table (MCSP, whose NP-hardness is open for single-output general circuits).

P07 (p. 625, Section 4.1, paragraph 1; Lean). Before: `provides non-linearity using an invertible affine transformation. This multiplicative inverse of the inverse of the affine transformation is used for decryption.`. After: `provides non-linearity through the multiplicative inverse in GF(2^8) followed by an invertible affine transformation. Decryption applies the inverse of the affine transformation followed by the multiplicative inverse.`. An affine transformation is linear up to a constant, so the non-linearity comes from the multiplicative inverse (FIPS-197 section 5.1.1), and the quoted decryption sentence is not grammatical or correct as written. *Lean:* `AES.sbox_table`, `AES.ginv_mul`, `AES.invAffine_affine`, `AES.affine_is_affine`, `AES.sbox_not_affine`, `AES.invSbox_sbox`.

P08 (p. 626, Section 4.2, paragraph 1; Lean). Before: `Toom-Cook (O(n^{log_k 2k-1}))`. After: `Toom-Cook (O(n^{log_k (2k-1)}))`. As printed the exponent reads log_k(2k) - 1 = log_k 2; Toom-k uses 2k-1 multiplications of size n/k, giving the exponent log_k(2k-1). *Lean:* `Arith.toomProds_eq`, `Arith.toom_between`, `Arith.toom_published_too_small`, `Arith.toom_witness`.

P09 (p. 627, Section 4.3, paragraph 2; Lean). Be-

fore: `increase the potential of the representation by order of 2^32 or 2^64.` After: `increase the potential of the representation by a factor of the order of 32 or 64 for bitwise operations and of 32^2 or 64^2 for schoolbook multiplication.` Replacing one-bit gates by w-bit word operations reduces the operation count by a factor polynomial in w, not 2^w. *Lean:* `Arith.words_exact`, `Arith.word_products`, `Arith.words_cover`, `Arith.word_gain_not_exponential`.

P10 (p. 629, Table 2, column C gmp; listing). Before: `Rows 128 and 256 both show 9.141ms (x86) and 7.640ms (x86-64); row 32 shows 851ms and 707ms, larger than row 64 (2.207ms, 1.015ms).` After: `Row 32: replace 851ms and 707ms by 851us and 707us (units). Row 256: the C gmp cells repeat the row-128 cells; they should read about 42ms (x86) and 35ms (x86-64), the row-128 values scaled by the ratio about 4.5 measured for the same code (an estimate; the exact cells need the machine of the other rows).`. The 256-bit cells duplicate the 128-bit cells, which is implausible, and the 32-bit cells exceed the 64-bit cells (the unit is microseconds, not milliseconds).

P11 (p. 627, Section 4.3 and Tables 1-2; listing). Before: `AES encryption with a fixed randomly generated key.` After: `AES encryption with the fixed key whose bytes are all 0x12.` The Nodes, Edges and Exec. N of Tables 1-2 are those of the graph built for the constant key 0x12...12, not for a randomly generated key; the AES graph size depends on the key bits.

P12 (p. 618, Section 1, paragraph 4; editorial). Before: `and the only resolution to structuring them into an abstract syntax tree (AST) involves introduction of variables and conditional expressions [30].` After: `and the resolutions to structuring them into an abstract syntax tree (AST) involve code duplication [27] or the introduction of variables and conditional expressions [30].` Section 2 (paragraph 7) states that restructuring is possible by code duplication [27] or by state variables [30, 31], so 'the only resolution' contradicts the paper itself.

P13 (p. 618, Section 1, paragraph 5; editorial). Before: `This study will take a look at specifically a case study where every control flow construct ... will be further embedded or converted into an irreducible loop nest ... The 2-SAT solver will be used to determine the exit conditions for all of the loops in question ... Performance measurements and a look at how powerful tools such as IDA Pro and Ghidra ... will be presented under Windows and Linux.` After: `State what the case study reports: Tables 1-2 measure plain Boolean-CFG implementations of AES and RSA, Table 3 a single hand-written irreducible-loop example; the transformation of Section 5 with 2-SAT-generated exit conditions is described but its application to AES and RSA is not measured in the paper.`. The announced case study (AES/RSA protected by irreducible loop nests with solver-generated exit conditions, measured on both platforms) is not what Sections 4-5 report. *Lean:* `Lang.aes_protected`, `Lang.modexp_protected`.

P14 (p. 631, Section 5, before Definition 4; Lean). Before: `removing extra back edges whose successor is not the inner-most non-header node.` After: `removing extra back edges whose source is not the inner-most non-header node.` Definition 4 keeps exactly the back edges (v_n, v_i), whose source (not successor) is the inner-most non-header node v_n; no back edge of M_n has v_n as successor.

P15 (p. 631, Section 5, before Definition 4; Lean). Before: `will have maximized multi-entry edges while minimizing all others.` After: `will have maximized multi-entry edges while minimizing the others up to one edge.` Among graphs with the maximum number of multi-entry edges the other edges number at least 2n-3, while M_n has 2(n-1); removing (v_n, v_1) keeps both irreducible-loop and multi-entry-edge counts.

P16 (p. 631, Section 5, Definitions 3 and 4; Lean). Before: `Definition 3 |MEE(K_n)| = ...; Definition 4 E_{M_n} = ..., V_{M_n} = v_i for all i in [1..n].` After: `Define MEE(G) for a CFG with a DFS, relative to the Havlak loop nesting forest of that DFS: an edge (x, y) with y in the body of a loop headed by w, y different from w, and x not a DFS descendant of w. State the counts |MEE(K_n)| and |E(M_n)| as claims about these definitions (they are not definitions), and write V_{M_n} = {v_i : i in [1..n]}.`. Multi-entry edge is never defined, Definitions 3 and 4 state counts rather than define objects, and V_{M_n} = v_i for all i is not a set.

P17 (p. 631, Section 5, paragraph on node splitting; Lean). Before: `However this optimization will still require a quadratic amount of code duplication for each multi-entry edge.` After: `However this optimization can still require an exponential amount of code duplication: for the complete graph K_n every reducible graph equivalent to it has at least 2^(n-1) nodes (Carter, Ferrante and Thomborson, POPL 2003).`. Node splitting on K_n is exponential, not quadratic, by Theorem 4 of Carter, Ferrante and Thomborson; the claim attributed to each multi-entry edge is unsupported. *Lean:* `Threat.one_variable_suffices`, `Threat.quadratic_is_edge_count`.

P17b (p. 617, Abstract (and Section 5, first sentence of paragraph 1 on p. 631); Lean). Before: `node-splitting or introduction of variables can have a quadratic complexity increase on the size of the control-flow graph.` After: `node-splitting can have an exponential and introduction of variables a quadratic complexity increase on the size of the control-flow graph.` Consequence of P17 for the abstract; the quadratic bound is the one the paper argues for the introduction of variables (DREAM).

P18 (p. 632, Figure 5(c); Lean). Before: `Graph (c): A has an exit edge (blue) and there is no injected condition node.` After: `Graph (c) with an injected condition node h: entry -> h, h -> A, A -> B, B -> h, h -> exit (blue), goto -> B (red), as in the code and in the text 'requires one node ... an increase of three edges'.`. In the drawn graph the loop can exit directly from A, so the path that executes A then B once does not exist (A, B, A is forced); the code and the text place the condition in a separate injected node. *Lean:* `Lang.fig5_traces`, `Lang.fig5c_cfg`, `Lang.drawn5c_no_AB`, `Lang.fig5c_cfg_AB`, `Lang.fig5c_irreducible`.

P19 (p. 633, Figure 6(c) and its caption; Lean). Before: `Graph (c) keeps the edge A -> B; caption 'or with an extra edge (c)'.` After: `Graph (c) without the edge A -> B (A returns to the condition, as in the code while (c && trueThenFalse) { labelA: A; } B;) and the caption 'with two extra edges (c)'.`. The code of (c) has no edge from A to B (control returns to the loop condition), and relative to (a) two edges are added (A -> c and the entry into A), not one. *Lean:* `Lang.fig6_traces`, `Lang.fig6c_cfg`, `Lang.fig6c_drawn_extra`, `Lang.fig6c_no_AB_edge`, `Lang.fig6c_irreducible`.

P21 (p. 634, Section 5.1, paragraph 4; editorial). Before: `differential computation analysis (DCA) and its subset differential power analysis (DPA).` After: `differential computation analysis (DCA), the`

software counterpart of differential power analysis (DPA). DCA (Bos, Hubain, Michiels, Teuwen, CHES 2016) applies the differential-power-analysis idea to software execution traces; neither attack is a subset of the other.

P22 (p. 634, Section 5.1, paragraph 4; Lean). Before: `It is resistant to differential computation analysis (DCA) and its subset differential power analysis (DPA) as the execution order changes based on the input.`. After: `It is conjectured to hinder differential computation analysis (DCA) when traces are not aligned by address, since the execution order changes based on the input; the scheme is not implemented or evaluated, and on address-aligned traces the same (vertex, value) pairs are observed as in the static scheme.`. No analysis or experiment supports resistance, and an input-dependent order of vertex executions does not change the set of intermediate values observable at fixed code addresses. *Lean:* `Threat.dca_last`, `Threat.dca_final_obs`, `Threat.dca_witness`.

P23 (p. 634, Section 5.1, paragraph 2; Lean). Before: `our proposed obfuscation scheme is indistinguishable from the underlying algorithm from a practical standpoint though not a theoretical one.` After: `our proposed obfuscation scheme computes the same function as the underlying algorithm but makes no claim of cryptographic indistinguishability, which is an open problem [8].` 'Indistinguishable from a practical standpoint' is not a defined security notion; what holds (and is proved) is functional equivalence. *Lean:* `Threat.correct`, `Threat.scheme_overhead`, `Threat.not_size_indist`.

P24 (p. 635, Section 5.1, paragraph 5; Lean). Before: `In the dynamic scheme, however, the repeated operations would require exponential time without proving these operations as repeated and removing them to reduce it to the static scheme.`. After: `In the dynamic scheme, however, repeated executions of the same vertex are observable in a trace; whether removing them to reduce the scheme to the static scheme is hard is not shown here.`. No argument or reference supports exponential time, and aligned traces expose which executions are repeats of the same vertex. *Lean:* `Threat.dca_repeats_visible`, `Threat.dca_witness`.

P25 (p. 634, Section 5.1; Lean). Before: `Section 5.1 as a whole.` After: see Appendix (full replacement of the section). Section 5.1 states security goals without an adversary model, assumption list or statement. *Lean:* `Threat.correct`, `Threat.imm_unsound`, `Threat.recover`, `Threat.one_variable_suffices`, `Threat.fault_class`, `Threat.dca_last`, `Threat.not_size_indist`, `Threat.nec_forcing`, `Threat.nec_stamped`, `Threat.nec_false_cells`, `Threat.nec_extra_respects`, `Threat.nec_schedule`, `Threat.nec_topo`.

P26 (p. 617, Abstract; Section 5 paragraph 1; editorial). Before: `limits in decidability as per Rice's theorem`. After: `static analysis of self-modifying code has limits in decidability (Rice's theorem and, for static analysis, Landi [18])`. Rice's theorem concerns extensional properties of programs; the cited reference for undecidable static analysis is Landi's alias-analysis result.

P27 (p. 628, Figure 3 code comments; listing). Before: `#2*2^n nodes, 2 predecessors per node; #2^n nodes, n predecessors per node.` After: `State the exact node counts of the two listed generators (Lean R11) or mark the comments as asymptotic.`. For BITSZ = 1 the generator decoder_shift_to_virtual creates 2^(m+1) + m - 4 nodes and decoder_to_virtual (m/2 + 2) 2^m nodes, so the comment '2^n nodes' for the latter is not its node count.

P28 (p. 627, Section 4.2, paragraph 3; Lean). Before: `the exponential growth of multiplication as the bit size gets to realistic cryptographic sizes.` After: `due to the super-quadratic (polynomial) growth of multiplication as the bit size gets to realistic cryptographic sizes.` The number of nodes of a Boolean multiplication circuit is polynomial in the bit size (schoolbook n^2, Karatsuba n^1.58), not exponential.

P29 (p. 618, Section 1 paragraph 5; Section 5.1 paragraph 3; Lean). Before: `will be beyond the scope of any state-of-the-art static analysis tools; Both schemes are practically resistant to static analysis, as the presence of dynamic SMC makes a static analyzer unable to safely analyze past the point that it is detected.` After: `will be beyond the scope of any state-of-the-art static analysis tools that treat the code as immutable; Both schemes are practically resistant to static analysis that treats code bytes as immutable.` Unqualified claims about all static analysis tools; the solver outputs are computable in linear time by an analyzer that models the self-modification, so the claim holds only for analyzers that do not.

T01 (p. 619, Typographical, p. 619; typographical). Before: `need not effect the implication graph.` After: `need not affect the implication graph.` Typographical error.

T02 (p. 619, Typographical, p. 619; typographical). Before: `|C_i|<=2.` After: `|C_i| \le 2.` Typographical error.

T03 (p. 626, Typographical, p. 626; typographical). Before: `Schonhage-Strassen.` After: `Sch\"onhage-Strassen.` Typographical error.

T04 (p. 626, Typographical, p. 626; typographical). Before: `elliptical curve digitial signature algorithm.` After: `elliptic curve digital signature algorithm.` Typographical error.

T05 (p. 626, Typographical, p. 626; typographical). Before: `multiplication.` After: `multiplication.` Typographical error.

T06 (p. 626, Typographical, p. 626; typographical). Before: `a performance penality.` After: `a performance penalty.` Typographical error.

T07 (p. 626, Typographical, p. 626; typographical). Before: `there performance will.` After: `their performance will.` Typographical error.

T08 (p. 634, Typographical, p. 634; typographical). Before: `reprsented as CFGs.` After: `represented as CFGs.` Typographical error.

T09 (p. 634, Typographical, p. 634; typographical). Before: `suspectible to such tools.` After: `susceptible to such tools.` Typographical error.

T10 (p. 630, Typographical, p. 630; typographical). Before: `gratuitious computations.` After: `gratuitous computations.` Typographical error.

T11 (p. 632, Typographical, p. 632; typographical). Before: `transfered to the first node.` After: `transferred to the first node.` Typographical error.

T12 (p. 631, Typographical, p. 631; typographical). Before: `if prefering to avoid.` After: `if preferring to avoid.` Typographical error.

T13 (p. 630, Typographical, p. 630; typographical). Before: `structured programing.` After: `structured programming.` Typographical error.

T14 (p. 629, Typographical, p. 629; typographical). Before: `a language whose syntax allows them is exceedingly rare.` After: `a language whose syntax allows them, is exceedingly rare.` Typographical error.

T15 (p. 628, Typographical, p. 628; typographi-

cal). Before: `\textbf{|xor\_to\_virtual|}`. After: `\textbf{\lstinline|xor\_to\_virtual|}`. Typographical error.

T16 (p. 633, Typographical, p. 633; typographical). Before: `if (!c): goto labelB;`. After: `if (!c) goto labelB;`. Typographical error.

T17 (p. 619, Odd-page running head; typographical). Before: `irreducible loopsand self-modifying code`. After: `irreducible loops and self-modifying code`. The odd-page running head lacks a space ('loopsand'); publisher typesetting.

B01 (p. 636, References, [21]; bibliographic). Before: `[21]: IEEE Transactions on Computers 137 (1988)`. After: `[21]: IEEE Transactions on Computers 37 (1988)`. Parhami 1988: vol. 37, no. 11, pp. 1470-1476, DOI 10.1109/12.8722 (the manuscript bib has volume 37)

B02 (p. 636, References, [26]; bibliographic). Before: `[26]: W. Tao, M. Jian, Z. Wei and C. Yu`. After: `[26]: T. Wei, J. Mao, W. Zou and Y. Chen`. given and family names swapped in the published list; the manuscript bib has Wei, Tao and Mao, Jian and Zou, Wei and Chen, Yu

B03 (p. 636, References, [8]; bibliographic). Before: `[8]: L. Buttyán and M. Horváth`. After: `[8]: M. Horváth and L. Buttyán`. ePrint 2015/412 lists M. Horváth first

B04 (p. 636, References, [19]; bibliographic). Before: `[19]: Annales Universitatis Scientarum Budapestinensis`. After: `[19]: Annales Universitatis Scientiarum Budapestinensis`. Latin plural of scientia

B05 (p. 636, References, [10]; bibliographic). Before: `[10]: Inform'atica`. After: `[10]: Informática`. the accent is typeset as an apostrophe

P30 (p. 625, Section 4.1, paragraph 1; Lean). Before: `Although the optimal one is NP-hard`. After: `Although finding a smallest circuit is not known to be tractable (and minimizing a given circuit is coNP-hard, see Section 4)`. Same conflation as P06: "the optimal one is NP-hard" is not established for synthesis from a truth table.

P31 (p. 627, Section 4.2, paragraph 2; Lean). Before: `non-adjacent form (NAF) and mutual opposite form (MOF) and their windowed forms which reduce the hamming weight of the exponent`. After: `non-adjacent form (NAF) and its windowed form which reduce the hamming weight of the exponent, and the mutual opposite form (MOF) which does not in general reduce it`. The MOF does not in general reduce the Hamming weight (5 = 101 in binary has weight 2 and MOF [-1,1,-1,1] of weight 4); only the NAF and its windowed form never exceed the binary weight. *Lean:* `Arith.mof_not_lighter`, `Arith.mofTotal_closed`, `Arith.wnaf_wt_le_popc`.

Appendix: replacement text for P25

Replace Section 5.1 by the following subsection (every statement is a Lean theorem; names in brackets).

5.1 Security. System: the original program is a structured program with goto over abstract blocks; the obfuscator is the static scheme of Figures 5 and 6 whose opaque conditions are outputs of the self-modifying 2-SAT solver on a planted formula; an image consists of the transformed program, the stamped formula and the placeholder bytes of the predicate cells. Adversary classes describe capabilities, not tools: Imm reads the predicate cells from the static code bytes; SmcSem implements stamping and the machine semantics; Struct restructures the control-flow graph; Trace and Dca see (address, value) pairs of a run; Fault replaces one written value. Assumptions: S1 the stamped formula is a planted formula whose extra implications respect the intended assignment; S2 the predicate cells the program reads are required variables whose intended value is false; S3 the circuit of the dynamic scheme is numbered topologically and its schedule has the final executions in dependency order; S4 only the capabilities above are considered. Results: T1 the image executes the blocks of the original program and the schedule yields the circuit's values [Threat.correct]; T2 an Imm analyzer is wrong on some image satisfying S1-S3 [Threat.imm_unsound]; T3 an SmcSem analyzer reports every predicate cell, and the Tarjan run on the stamped formula costs at most 2|V|+|E| steps in the clean cost model [Threat.recover], so opacity is relative to analyzers without the machine semantics and is not computational hardness; T4 one state variable always suffices to flatten a control-flow graph, and the quadratic quantity of Section 2 is the multi-entry-edge count of K_n, not a cost of restructuring [Threat.one_variable_suffices, Threat.quadratic_is_edge_count] (an exponential lower bound for node splitting is due to Carter, Ferrante and Thomborson and is not proved here); T5 a fault in a non-final execution does not change the output, a fault in a final execution does [Threat.fault_class, Dyn.fault_final_changes]; T6 on address-aligned traces every dependency-respecting schedule gives the same (vertex, final value) pairs, and repeated executions of a vertex are visible [Threat.dca_last, Threat.dca_final_obs, Threat.dca_repeats_visible]; T7 the scheme is not an indistinguishability obfuscator for the number of vertices [Threat.not_size_indist]. Each assumption is necessary for the theorem it supports [Threat.nec_forcing, nec_stamped, nec_false_cells, nec_extra_respects, nec_schedule, nec_topo]. Not claimed: hardness of the predicates against an analyzer with SmcSem, resistance to any existing tool, any indistinguishability or virtual-black-box property.